

VeriCon: Mining, Falsifying, and Injecting Configuration Constraints for LLM Inference Engines

Shuheng Zhao

Abstract—Hand-written configuration knowledge does not last: of the three “known constraints” a production tuner (SCOOT) ships, audited release by release at source and execution level, one was removed upstream, one is now dormant under changed defaults, and six of six tested folk rules are unenforced on a second engine. We present VeriCon: it mines parameter constraints from engine source with LLM agents (verbatim `file:line` evidence each), *falsifies* them by executing the engine’s real validation path on violating configurations (no GPU), and injects the verified set into a Bayesian tuner. On vLLM v0.30.0 it extracts 716 constraints, confirms 113 via the construction path, and measures 71.9% recall (87.2% at ± 1 line; 84.8% equal-weight under an independent human labeler; 53.0%/63.2% design-weighted); on SGLang v0.5.20, 420. Across 8,000+ simulated trials, constraint injection cuts tuning cost by 43.5–50.9%; the manual and mined arms coincide by construction in that subspace, so this isolates the value of *constraint knowledge*—*automatic extraction*’s value is established in RQ1–RQ4 and on the real engine. A 400-trial llama.cpp micro-study separates the sources: 0% invalid trials for source-mined constraints vs 5% (online learning), 12% (folk rule), 17% (none), at 13–34% lower cost—all on a four-core laptop at zero cash cost.

Index Terms—LLM inference, configuration tuning, configuration constraints, source analysis, LLM agents, falsification.

I. INTRODUCTION

LLM inference services are deployed with engines such as vLLM [1], SGLang [2], and TensorRT-LLM, and their service-level objectives (TTFT, TPOT, tail latency, throughput) are strongly affected by engine configuration. SCOOT [3] showed that tuning a handful of parameters can reduce TTFT by up to 98.9% and TPOT by up to 49.9% relative to defaults—while noting that merely nine parameters form a search space of “over one hundred billion configuration points.”

The constraint problem. Large configuration spaces contain many *illegal* regions: parameter combinations that crash the engine at startup or during stress testing. Configuration issues are a major failure class in practice: a recent study of 929 real bugs in five LLM inference engines attributes 23% of them to configuration [4]. SCOOT formulates tuning as constrained optimization and distinguishes two mechanisms:

- **Known constraints**—cross-parameter rules such as `max_num_batched_tokens >= max_num_seqs`, which “can be provided to the tuner ahead of time” [3]; and

- **Hidden constraints**—service-specific crash combinations “initially unknown and must be learned throughout the tuning process” [3].

Both mechanisms have structural limitations. Known constraints are **hand-written**: SCOOT’s open list contains **three rules per engine**. Hidden constraints are learned **online**, i.e., by paying for crashes. Neither is **maintained across engine versions**—a problem SCOOT itself acknowledges: “inference engines update frequently ... which makes the modeled parameter-performance relationship outdated” [3].

This paper’s observation. Executing the engines themselves, we find that the deeper problem is not that constraint knowledge is scarce, but that it *decays*: hand-written rules rot as engines evolve, and even well-meaning folk knowledge is often simply wrong. Concretely:

- 1) SCOOT’s three hand-written rules do not survive scrutiny: **one was a real runtime-enforced constraint in the anchor versions that upstream later removed** (v0.4.2/v0.5.5 raise at runtime when a prefix-cache hit occurs under chunked prefill, `worker/model_runner.py`; the guard is gone from v0.6.6 onward—construction-time checks never rejected the combination), and **one lost its practical force** as engine defaults changed (the `max_num_batched_tokens >= max_model_len` rule is conditional on chunked prefill being disabled, which V1 enables by default).
- 2) Folk wisdom fails too: on llama.cpp [5] v0.5.0, six widely assumed rules (e.g., `n_ubatch <= n_batch`) are **not enforced** by the engine.
- 3) Conversely, the engines contain *far more* verifiable constraints than any manual list: vLLM v0.30.0 exposes **259 distinct CLI options** (262 registrations), and its source embeds hundreds of cross-parameter checks.

Approach. We present VeriCon, an *extract–falsify–inject* pipeline (Figure 2):

- **Extract.** Static analysis enumerates parameters and validation sites; LLM sub-agents translate code-level checks into parameter-level constraints, each carrying verbatim source evidence (`file:line` + snippet). No evidence, no constraint.
- **Falsify.** Each candidate is compiled into a *violating configuration* and executed against the engine’s real validation path (no model loading, no GPU). A constraint enters the verified set only if violating it actually fails.

- **Inject.** Verified constraints become sampling bounds for a Bayesian tuner; their value is quantified against unconstrained and manual baselines in a controlled simulation study.

Contributions.

- 1) **A measured decay of hand-written constraint knowledge.** We audit the constraint knowledge that a production tuner ships, release by release and at both source and execution level: **two of SCOOT's three published rules have lost their force in current releases**—one runtime guard was removed upstream; one keeps its semantics but was rendered dormant by changed defaults—while one remains stable across five releases, and 6 of 6 tested folk rules are unenforced on a second engine. The same audit quantifies what replaces it: 716 mined constraints (for the engine's full configuration surface) vs 3 manual rules (for a 9-parameter tuning subspace).
- 2) **A source-grounded extraction method.** An LLM + static-analysis pipeline yields **716 constraints** for vLLM (100% source evidence) and **420 for SGLang**; two independent audits (40 constraints, different seeds) find **no incorrect constraint**; two stratified gold-standard audits (300 sites) measure **71.9% recall (87.2% within ± 1 line; 53.0%/63.2% under constraint-level design weighting)**; an independent non-scanner frame gives a *lower*, scope-different estimate (68.0% at ± 1 line, vs. 87.2% on the candidate-pool frame); and an independent run with a different LLM re-discovers **97.2%** of the core set (evidence-anchored criterion), contributing 34 additional candidates; a third blind labeling pass over 150 sites ($\kappa = 0.65$, released kit protocol) and an **independent human labeler** on the same sites ($\kappa = 0.53$ vs evaluator, 0.68 vs third pass; 84.8% equal-weight ± 1 capture) reproduce the recall estimate.
- 3) **Execution-based falsification at scale.** Without GPUs, we test 155 constructible constraints against vLLM's real validation path and **confirm 113** (111 primary + 2 cross-model); 22 fail to trigger at construction (cross-config/runtime) and 22 are blocked by construction dependencies. The extraction protocol transfers to **SGLang (420 constraints)** and the falsification protocol to llama.cpp (2 confirmed, 6 refuted).
- 4) **Measured tuning benefit.** In **8,000+ simulations** (three-arm study, dimension scaling, arrival-rate and request-length workloads, sampler-in-loop runs, and a SCOOT-style online-learning arm), injecting constraint *knowledge* cuts cumulative cost by **43.5–50.9% across spaces (2-D–6-D), 48.3–49.6% across arrival rates, and 46.2–48.3% across three request-length workloads** (headline: 48.6% with CI [46.7, 50.0])—the two constrained arms coincide by construction in this simulator subspace, so this measures the value of constraint knowledge, not of automatic extraction (that is RQ1–RQ4 and the real-engine study below)—halves trials-to-target, drives charged invalid-config sampling from **31.5% to 0%**, does not reduce final quality, and stays **20–23% cheaper than online constraint learning**. A 400-trial real-engine

micro-study on llama.cpp (four arms, six dimensions, 1.5B model) separates the arms: **0% invalid trials** for source-mined constraints vs **5%** (online learning), **12%** (folk rule), and **17%** (none), at **13–34% lower cost**.

- 5) **Reproducible artifacts.** Constraint graphs (with evidence and validation status), falsification harnesses for two engines, the tuning harness, and all raw records—produced entirely on a four-core CPU laptop at zero cash cost.

II. BACKGROUND AND MOTIVATION

A. The configuration space of LLM inference engines

Modern engines built around continuous batching [6] expose large, evolving parameter surfaces spanning batching and scheduling [7], parallelism, and disaggregation [8], [9]; see [10] for a survey. In vLLM v0.30.0 (commit ced6857) we statically enumerate **9,670 annotated configuration fields** across 2,258 classes, of which **585** live in the dedicated `vllm/config/` layer, **242** in `EngineArgs`, and **262 CLI-flag registrations** (259 distinct flag names) are found. Constraints arise from four mechanisms: pydantic field bounds (`ge/le`), enum literals, `__post_init__`/model-validator checks (`if cond: raise`), and assertions.

B. Why constraints matter for tuning

A tuner that samples the raw space pays for every illegal point. In our controlled study (§IV-F), unconstrained Bayesian optimization [11] was charged for constraint-violating configurations **31.5% $\pm$ 5.1%** of the time (the theoretical log-uniform sampling probability of the illegal region is $\approx 2/3$; the optimizer learns to avoid it partially over trials). Each illegal trial costs engine startup and stress-test time—SCOOT reports 5–10 minutes per evaluation [3], and a crash wastes that budget just as a successful run does. Constraints are therefore not a cosmetic pruning device: they are a **budget multiplier**. Misconfiguration is likewise a recognized source of technical debt in ML systems [12].

C. Why existing constraint sources are insufficient

Notably, cDep observes that existing NLP-based analyses do not consider dependencies *between multiple configuration parameters*, and calls for NLP to discover new types of configuration dependencies [18]—a call our LLM-based extraction answers.

D. Version drift is real: an execution study of SCOOT's rules

We constructed violating configurations for SCOOT's three vLLM rules and ran vLLM v0.30.0's validation path (Figure 1 traces each rule's lifecycle across the six releases we scanned; source-level on all six, execution-level on v0.5.5, v0.10.0, and v0.30.0):

On llama.cpp v0.5.0 we executed 17 test cases covering seven candidate rules: two real constraints were confirmed (`n_batch >= 1` aborts with SIGABRT when violated; quantized KV cache requires flash attention), while six

TABLE I
EXISTING CONSTRAINT SOURCES AND THEIR LIMITATIONS.

Source	Form	Limitation
SCOOT [3]	3 hand-written rules per engine (scoped to a 9-parameter tuning subspace)	Careful, but hand-written; decays with versions (measured in §II-D)
SCOOT hidden constraints [3]	Online random-forest feasibility model	Learned by crashing; per-service; no source semantics; not transferable
PerfSense [16] / CongSense [17] / cDep [18]	LLM classifies parameters as performance-sensitive	Binary labels, not relations; not executable
	Static dependency extraction (Java/Scala bytecode)	Non-LLM; pattern-driven; dependencies are facts for developers, not executable pruning rules
Vidur-Search [19]	Simulator-side grid enumeration	Constraint space hard-coded in a human-written simulator, not mined from a real engine

TABLE II
SCOOT’S THREE HAND-WRITTEN RULES (AS PUBLISHED FOR V0.4.2/V0.5.5 [3]), RE-CHECKED AT SOURCE AND EXECUTION LEVEL.

SCOOT rule	Source + execution re-check	Verdict
mtib $\geq$ max_num_seqs	present verbatim in v0.5.5 (config.py:938--942) and v0.30.0 (scheduler.py:309); execution-falsified	stable across five releases
chunked-prefill $\perp$ prefix-caching	runtime guard in v0.4.2 (worker/model_runner.py:254--258) and v0.5.5 (502--504): raises on a prefix-cache hit under chunked prefill; removed from the main runner in v0.6.6+ after upstream added support	enforced at the anchor versions; removed later (decay)
no chunked-prefill $\Rightarrow$ mtib $\geq$ max_model_len	conditional check already in v0.5.5 (config.py:927--936), unchanged in v0.30.0; V1 enables chunked prefill by default	same semantics; lost practical force

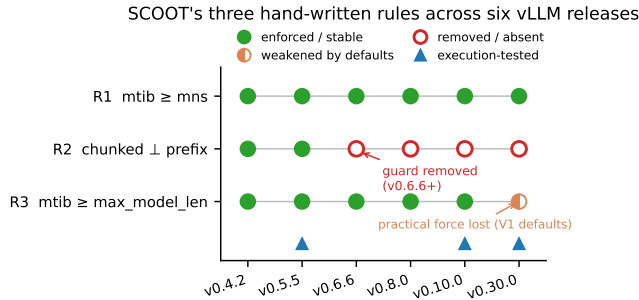


Fig. 1. The lifecycle of SCOOT’s three published rules across six vLLM releases, re-checked at source and (on the marked versions) at execution level. Hand-written constraint knowledge decays: one rule was removed upstream, one survives only nominally.

assumed rules were not enforced, including `n_ubatch  $\leq$  n_batch`.

Takeaway. Constraints must be *mined from source* and *verified by execution*, and re-mined as engines evolve.

III. METHOD: EXTRACT-FALSIFY-INJECT

A. Problem formulation

Let $P = \{p_1, \dots, p_n\}$ be the engine’s configuration parameters with domains D_i (integer/real ranges, enumerations, booleans, structured fields); the *configuration space* is $\Lambda = \prod_i D_i$ and an assignment $\sigma \in \Lambda$ is one launch configuration. A *constraint* is $c = (id, t, \varphi, \pi)$ —a type t , a predicate φ over a parameter subset $\pi \subseteq P$, and provenance. Seven types cover the engines we study: **inequality** ($x \geq y$), **range** ($x \in [l, u]$), **enum** ($x \in S$), **arithmetic** ($x \bmod y = 0$, $x = c \cdot y$), **implication** ($A \Rightarrow B$), **mutual exclusion** ($\neg(A \wedge B)$), and **structural type**. An assignment *satisfies* c ($\sigma \models c$) iff φ

evaluates true under σ ; the *legal domain* of a constraint set C is $\Lambda_C = \{\sigma \in \Lambda : \forall c \in C, \sigma \models c\}$ and the *nominal pruning ratio* is $\rho_{\text{grid}} = 1 - |\Lambda_C|/|\Lambda|$. A *constraint graph* is $G = (P, C)$ with evidence and validation status per node. Our goal is to produce G with evidence, falsify it by execution, and measure the effect of sampling from Λ_C rather than Λ .

B. Parameter inventory (static)

We parse the engine tree with Python’s AST and extract every class-level annotated assignment (name, type, default, Field metadata), plus every CLI flag registered via `add_argument`. Each entry stores file, line, and the source snippet. For vLLM v0.30.0 this yields 9,670 fields and 262 flags; for downstream filtering we keep the `vllm/config/` and `EngineArgs` subsets (585 + 242).

C. Candidate generation (static)

We scan for validation sites of two syntactic forms: `assert <cond>` and `if <cond>: raise ...`. On vLLM v0.30.0 this produces **13,301 candidates** (8,498 assertions; 4,803 guarded raises). We tier candidates by path (`vllm/config/` > `engine/core` > `rest`) and keep those whose condition or message mentions at least one parameter name from the inventory, reducing the agent workload to **3,129 candidates** (363 in the two high-priority tiers).

D. Agent-based structuring

We partition the high-priority candidates by module and assign one LLM sub-agent per partition (deepseek-v4.1-flash; five partitions, 50–94 candidates each). Each agent re-reads the source around every candidate (± 30 lines) and emits constraints in a fixed schema: `type, expr` over real field

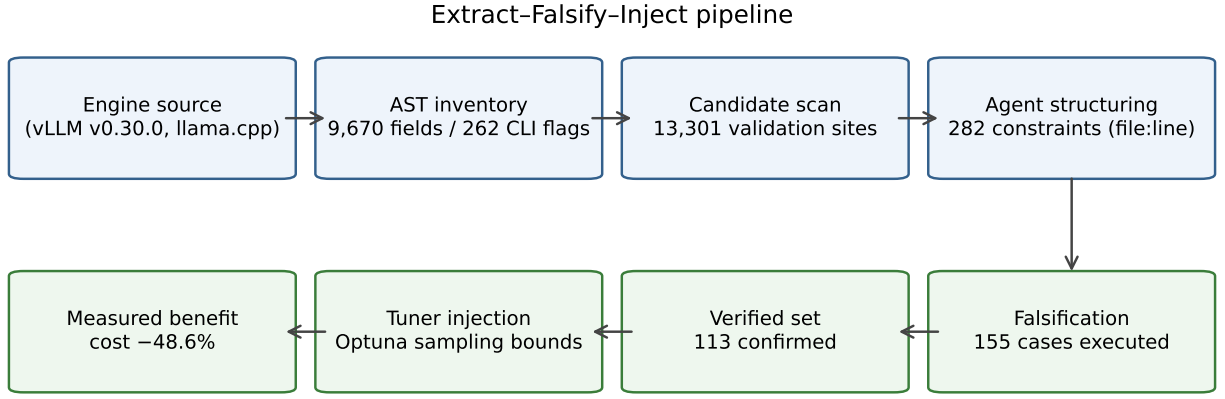


Fig. 2. The extract-falsify-inject pipeline with measured numbers at each stage (9,670 fields $\rightarrow$ 13,301 validation sites $\rightarrow$ 282 constraints in the core tiers, 716 with the full-coverage extension $\rightarrow$ 155 executed cases $\rightarrow$ 113 confirmed $\rightarrow$ tuning cost -48.6%).

names, params, severity, a one-sentence mechanism, and evidence[] with verbatim snippets. Candidates that are internal invariants, hardware checks, or non-parameter state are dropped with a recorded reason.

A merge stage applies four mechanical checks: (R1) every parameter exists in the inventory; (R2) every evidence line exists; (R3) every snippet appears verbatim on its line; (R4) the expression is non-empty and references declared parameters. Violations are rejected, not silently dropped. **Result: 282 constraints across 26 scopes, 100% with evidence, zero mechanical rejections** (after fixing one CLI-destination derivation bug that had initially mis-rejected two legitimate constraints).

Cross-model replication. Re-running the identical pipeline on a second LLM (GLM-5.3-Flash, vs. deepseek-v4.1-flash for the primary run; same inputs, schema, and rules) yields 320 constraints. Under the strict evidence-anchor criterion (shared file:line), **97.2% of the primary run's 282 constraints are independently re-discovered**; conversely, 88.1% of the second run's constraints are covered by the first, and the second run contributes 34 additional candidates (21 error-level). Single-model extraction still has blind spots (8 constraints found only by the primary run), which motivates releasing the union as the artifact.

E. Falsification by execution

For each constraint we construct a *violating assignment*—falsification by construction, in the spirit of property-based testing [20]. Four LLM sub-agents translate constraints into concrete field assignments (e.g., kv_connector is not None $\rightarrow$ kv_role is not None becomes {"kv_connector": "NixlConnector", "kv_role": null}), skipping constraints whose checks occur at request time or in runtime components. The executor then instantiates the engine's real configuration classes with a **baseline-first protocol**: a legal baseline must construct successfully, after which the violating override is applied; a constraint is *confirmed* only if the override triggers an error.

This guards against counting unrelated construction failures as evidence.

vLLM v0.30.0 specifics (documented for reproducibility): configurations are pydantic models whose dataclasses.fields expose FieldInfo defaults; SchedulerConfig requires “virtual” keys (max_model_len, is_encoder_decoder) consumed by a before-validator; nested configs must be built as sub-objects; ModelConfig can be constructed offline from a local tiny model. We validate 155 constraints this way (§IV-C). For llama.cpp we execute the real CLI binary and capture exit codes and stderr.

Witness-level confirmation (Proposition 1). If the protocol confirms c on version V , then V 's validation path rejected the constructed violating witness. If the validator is *total* over π (it rejects *every* violating assignment, not a subset), confirmation extends to all protocol-reachable assignments sharing the tested code path. We state this as a proposition, not a universal guarantee: a validator may guard only some branches—our adversarial audit found three branch-precondition omissions among 30 nodes—and the evidence covers only (i) protocol-reachable construction paths, since cross-configuration and runtime constraints remain not-tested (§IV-C), and (ii) the tested version, since constraint knowledge drifts (§IV-D). Every graph node therefore carries a version and a validation status.

F. Injection into the tuner

Generic constrained sampler. We inject C as *derived sampling domains* rather than hard-coded bounds. For a parameter x and an already-sampled assignment σ' , the reachable domain is $R(x \mid \sigma', C) = D_x \cap \bigcap_{c \text{ evaluable}} \text{bound}_c(x, \sigma')$, where bound_c raises or lowers integer bounds ($x \geq y \Rightarrow \text{lo}(x) \leftarrow \max(\text{lo}(x), y)$; $x \geq c \cdot y$; $x < y \Rightarrow \text{hi}(x) \leftarrow y - 1$), restricts to multiples ($x \bmod y = 0$), or filters enumerations. The sampler walks parameters in dependency order and samples within R . The implementation reads the graph JSON directly—a data-driven injection, demonstrated on a six-dimensional space in §IV-F.

TABLE III
EXPERIMENTAL SETUP.

Component	Details
Engines	vLLM v0.30.0 (commit <code>ced6857</code> ; configuration-construction validation, no GPU); llama.cpp 0.5.0-dev build 11194 (real execution, CPU); Vidur [19] (MLSys'24 simulator, vLLM-V0 semantics) for tuning
Hardware	One laptop: Intel i5-7300HQ (4 cores), 23.7 GB RAM; the installed GTX 1050 (CC 6.1) is unsupported by vLLM, so all experiments are CPU-only. Cash cost: ¥0
Model (anchor)	Qwen2.5-0.5B-Instruct (anchor grid) and Qwen2.5-1.5B-Instruct (micro-study), Q4_K_M, for llama.cpp
Tuning spaces	S5: 2-D/5-D ($mns \times mtib \times block_size \times watermark$; $block_size$ fixed to 16 by the simulator). Dimension scaling adds explicit <code>num_blocks</code> (6-D, with the KV-capacity rule) and workloads at $qps \in \{8, 12, 16\}$
Reproducibility	Every artifact records engine commit, configuration, metrics, and cost; scripts and raw records released (§VIII)

Zero-violation sampling (Proposition 2). The sampler never returns a violating assignment: at each step it samples inside the intersection of all currently evaluable bounds, and reports failure (no sample) when that intersection is empty. It *succeeds* whenever every step's feasible domain is non-empty—an assumption, not a guarantee: the 716-node graph contains 7 cyclic implication clusters (largest cycle 3), for which no linear dependency order exists and bounds are instead computed from constraint evaluability as parameters are assigned. No empty-domain failure occurred in the 90 sampler-in-loop runs (§IV-F).

From coverage to cost. With n trials, violation rate r , and invalid-trial cost c_{inv} , the expected cost is $n(1 + c_{inv}r)$, so constraining $r_b \rightarrow r_a$ saves $c_{inv}(r_b - r_a)/(1 + c_{inv}r_b)$; the saving exceeds 30% iff $r_b \geq 3/(7c_{inv})$ (e.g., $r_b \geq 14.3\%$ at $c_{inv} = 3$). Crucially, r must be computed under the *tuner's sampling distribution*: the nominal grid ratio ($\approx 12\%$ in our two-dimensional space) badly underestimates the waste of log-uniform sampling (83.3% for the six-dimensional space, analytically 66.7% from the token-budget constraint and $\approx 50\%$ from the KV-capacity rule); an adaptive sampler (TPE [21], via Optuna [22]) partially mitigates it (31.5% measured), and constraint injection removes it entirely (0%) (Figure 9). §IV-F verifies the analytical saving (48.6%) against the measured value.

IV. EVALUATION

A. Setup

B. RQ1: How good is the extraction?

Static layer: 9,670 annotated fields; 262 CLI flags; 13,301 validation candidates; 3,129 parameter-related; 363 in high-

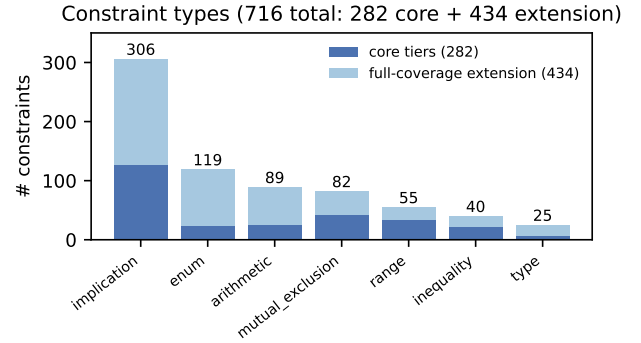


Fig. 3. Constraint type distribution: core tiers (282) and full-coverage extension (434), by type.

priority tiers.

Agent layer: 282 constraints from the configuration/engine tiers (implication 127, mutual exclusion 42, range 33, arithmetic 26, enum 24, inequality 22, type 8; Figure 3 shows both tiers; top scopes `vllm_config` 61, `speculative` 36, `parallel` 27). Extending coverage to all 3,129 parameter-related candidates (10 additional agent batches over the remaining 2,766 sites) adds 434 more constraints, for **716 total**, all with verbatim source evidence.

Audits. A seeded random sample of 20 constraints was checked against source line-by-line by the pipeline operator: **16 fully correct, 4 partially precise, 0 incorrect**. An independent second batch (different seed; reviewer role) scored **20/20 correct**. Across both batches (40 constraints, two independent samples), **no constraint was found incorrect**. A further **adversarial** audit (30 constraints; a fresh instance explicitly instructed to refute them) also found **0 incorrect**; its three partial findings are branch-precondition omissions, now recorded as `audit_notes` in the released graph. Cumulative: **130 constraint-audits, 0 incorrect**. All audits in this paper were produced by pipeline roles or independent AI instances; for the 150-site labeling kit, an independent human labeler (a technically literate non-expert following the released instructions) has since repeated the pass: agreement with the evaluator labels is 75.3% raw / $\kappa = 0.53$ and with the third pass 83.9% / $\kappa = 0.68$ —comparable to the AI-vs-AI agreement—and the human labels reproduce the capture estimate (84.8% equal-weight and 72.2% constraint-weighted ± 1 line); the human is again stricter than the evaluator, the same direction as every independent pass.

Recall (gold standard). Two independent stratified samples (300 sites total; 50 per tier per sample, fixed seeds; population tier sizes 244/119/2,766) were labeled against source by the evaluator: 203 true constraints and 97 non-constraints. Under *equal weighting across tiers*, the pipeline captures **71.9%** of the true constraints by exact evidence line and **87.2% within ± 1 line** (Wilson 95% CI [65.4, 77.7] and [81.9, 91.1]); because tier-3 sites dominate the population (88.4%), the site-level *design-weighted* estimates (weights = candidate count per tier, 244/119/2,766) are **50.8% (exact) and 59.5% (± 1)** (95% CI $\approx$ [42.1, 59.5] and [50.9, 68.1]); weighting instead by the estimated number of true constraints per tier (the estimand

for “recall over all true constraints”) gives **53.0% (exact) and 63.2% (± 1)**—we report all three estimands and quote the equal-weight one in the abstract. The intervals treat the stratified samples as simple random; with tier 3 dominating the population they are approximate. By tier (exact/ ± 1): 66%/96% (config), 94%/96% (engine/core), 48%/55% (model/kernel); 5 of 97 non-constraint sites carry graph evidence. Recall is line-level (does the graph anchor the labeled site?); expression-level correctness is audited separately above. An independent blind labeling pass (a different model instance; 120 audited sites; identical criteria) agrees at 80.0% raw / $\kappa = 0.60$ (per batch 0.75 and 0.46); 23 of 24 disagreements are in the direction of the evaluator being more inclusive, and they span internal-invariant *and* configuration-layer checks. On the batch-2 60-site subset, ± 1 -line recall is 79.1% under evaluator labels and **92.3%** under the stricter independent labels, so the sensitivity concentrates in the disputed sites. A third independent pass over 150 of the same sites—executed blind through the released labeling kit (site material only; no capture information; fresh-context instances)—agrees with the evaluator labels at **82.6% raw** / $\kappa = 0.65$ and yields **88.3% equal-weight and 75.3% design-weighted ± 1 -line capture** under its stricter labels (exact: 66.7%/48.4%).

An independent, non-scanner frame. To test whether this estimate is an artifact of the candidate funnel, we drew a separate frame without using any scanner output: 150 of the 2,479 files in `vllm/` (fixed seed), with every `assert/raise` site enumerated by an independent AST pass; a name table of engine-configuration fields pre-filtered the 98 strongest candidates, which were then adjudicated one by one by a single adjudicator (the pipeline operator). Fifty are engine-configuration validation sites, of which the released graph captures **34 by ± 1 line (68.0%, Wilson 95% CI [54.2, 79.2]; 24 exact-line)**—a lower estimate than the candidate-pool frame under the same ± 1 criterion (87.2%, CI [81.9, 91.1]; the intervals do not overlap), which we read as a *scope* difference rather than a corroboration: the independent frame samples the whole source tree, the funnel frame the scanner’s candidates; the 16 residuals sit in attention-backend quantization/dtype combinations, per-model TP divisibility checks, and runtime-state modules (`forward_context`, `hispase`) outside the tier plan. The frame also shows that most `assert/raise` sites in the tree are not configuration constraints at all (kernel, tensor, and checkpoint checks)—the engine-configuration surface is a small, identifiable slice, and §VI states its boundaries explicitly.

Cross-model robustness (§III-D): 97.2% evidence-anchored re-discovery by an independent LLM run.

C. RQ2: Do the constraints hold in the real engine?

155 constraints were constructible; 127 (45%) were classified as runtime/request-time/environmental and not constructible offline—this is the construction-path protocol’s boundary, not a correctness claim (SCOOT’s rule #2 is exactly such a runtime guard, §II-D).

Confirmed constraints include the flagship rule mined automatically from source: `mtib >= mns  $\rightarrow$  ValueError:`

TABLE IV
FALSIFICATION OUTCOMES (vLLM v0.30.0, 155 EXECUTED CASES).

Status	Count	Meaning
confirmed (primary)	111	violating assignment triggers a real engine error
confirmed (cross-model)	2	same protocol, second LLM batch
no_error	22	baseline constructs; violation not triggered at construction
blocked	22	construction dependencies (heavy configs)
total executed	157	155 constructible primary cases + 2 cross-model cases

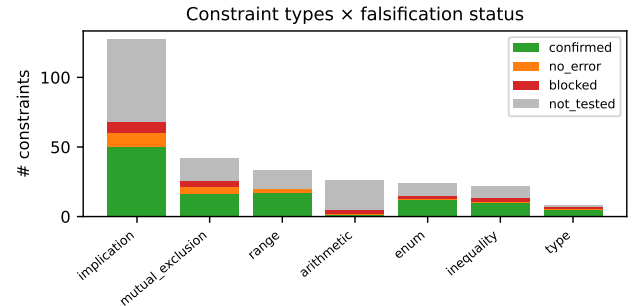


Fig. 4. Confirmation status by constraint type.

`max_num_batched_tokens` (16) must be greater than or equal to `max_num_seqs` (32) (`vllm/config/scheduler.py:309`), plus 23 parallel-topology rules, 16 speculative-decoding [23] rules, 10 profiler rules, and 8 compilation rules (Figure 4).

D. RQ3: Do hand-written constraints survive version drift?

No—see §II-D. Of SCOOT’s three rules: **one is stable across five releases** (execution-confirmed on v0.5.5, v0.10.0, and v0.30.0); **one was a runtime-enforced constraint in the anchor versions that upstream later removed** when it added support for the combination (v0.4.2/v0.5.5 `worker/model_runner.py` raises on a prefix-cache hit under chunked prefill; the guard is absent from the main runner in v0.6.6 onward—and construction-path checks never rejected the combination, which is why our construction tests at v0.5.5/v0.10.0/v0.30.0 all pass); **one keeps identical semantics but lost practical force** because V1 enables chunked prefill [7] by default. On `llama.cpp`, 2 of 17 executed configurations confirmed real constraints and 6 assumed rules were not enforced. **Hand-written constraint sets decay; and runtime guards sit outside construction-path verification—automatic extraction with execution verification is necessary.**

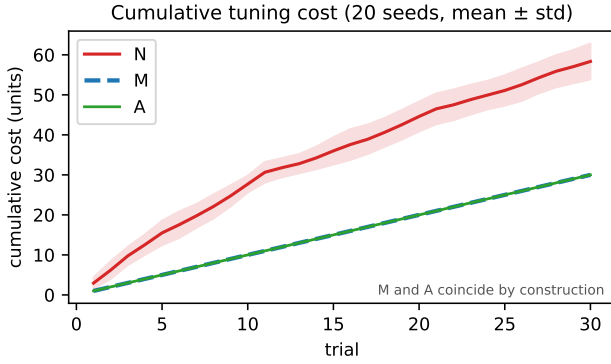


Fig. 5. Cumulative tuning cost (S5 three-arm study, 20 seeds).

E. RQ4: Does the protocol transfer across engines?

The same extraction protocol applied to **SGLang v0.5.20** (a different configuration architecture: ServerArgs validation hooks) yields **420 constraints** with 100% source evidence and zero mechanical rejections; the constraint-family profile mirrors vLLM’s—KV-cache/memory, parallelism/topology, quantization, and speculative decoding dominate. A quality audit on a type-stratified sample of 40 constraints (plus a blind re-audit of 20 by an independent pass) found **0 incorrect constraints** (60 constraint-audits); the single disagreement—a path exception not recorded in the expression—was fixed in the released graph.

The falsification protocol applied to llama.cpp (a C++ engine) confirmed two constraints and refuted six (§II-D). Real measurements over 24 configurations (threads $\in \{2, 3, 4\} \times$ batch $\in \{128, 256, 512\} \times$ ubatch $\in \{64, 128, 256\}$; 5 repetitions each) show prompt-processing up to **159.8 tok/s** and generation up to **45.9 tok/s** on the 0.5B model—establishing that the method is not vLLM-specific and that our records capture real, not simulated, behavior.

F. RQ5: What is the tuning benefit?

Setup. Vidur [19] (vLLM-V0 semantics), LLaMA-2-7B on A100, trace `splitwise_conv` [24] (19,366 requests), 384 requests per run, Poisson arrivals at qps = 12.0. We calibrated the workload first: the system’s capacity is ≈ 9.4 rps; at qps = 8 the objective is arrival-limited and non-discriminative, whereas at qps = 12 representative configurations span goodput **0.79–8.96**. SLO: TTFT-p90 ≤ 1.0 s and TPOT-p90 ≤ 0.1 s. Search space: `max_num_seqs` $\in [16, 256]$ (log) $\times$ `max_num_batched_tokens` $\in [16$ or `max(seq, 1024), 8192]` (log) $\times$ `watermark` $\in [0.001, 0.2]$ (log); `block_size` is fixed to 16 because the simulator’s bundled profiling data only supports 16 (a simulator limitation, not a real-engine constraint).

Arms. **N** no constraints; **M** manual (SCOOT-projected: `mtib` $\geq \max(\text{mns}, 1024)$, i.e., rule #1 plus the no-chunked-prefill rule #3 in V0 semantics); **A** automatic (mined graph projected); **S** SCOOT-style (only the manual rule, with online boundary learning from failures). Each arm: 20 seeds $\times$ 30 trials = 600 trials; 1,800 simulations in 238 minutes.

TABLE V
HEADLINE THREE-ARM STUDY (20 SEEDS $\times$ 30 TRIALS PER ARM).

Arm	Cost (units)	Invalid rate	Best goodput	Trials-to-target
N	58.350 $\pm$ 4.614	31.5% $\pm$ 5.1%	8.954 $\pm$ 0.027	9.80 $\pm$ 5.57
M	30.000 $\pm$ 0.000	0%	8.987 $\pm$ 0.029	5.35 $\pm$ 2.66
A	30.000 $\pm$ 0.000	0%	8.987 $\pm$ 0.029	5.35 $\pm$ 2.66

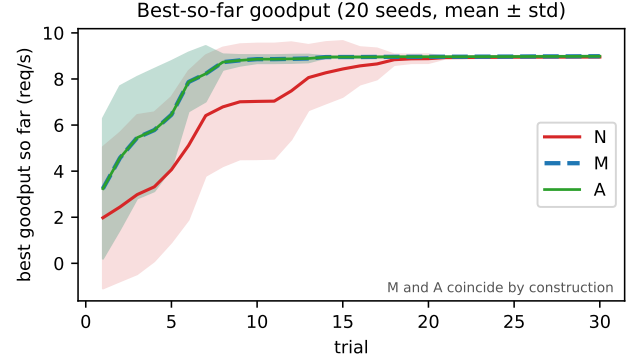


Fig. 6. Best-so-far goodput (S5).

Findings. Cost -48.6% by ratio of means (paired per-seed reduction 48.3%, 95% CI [46.7%, 50.0%]); trials-to-target -45.4% (N: 9.80 [7.45, 12.25] vs. M/A: 5.35 [4.15, 6.45]; target = $0.9 \times$ the pooled best goodput, defined here for the first time—no censoring occurs, max 21/30); charged invalid sampling 31.5% $\rightarrow$ 0%; final quality not reduced (8.954 $\rightarrow$ 8.987, paired difference +0.033). We designate the S5 N $\rightarrow$ M cost reduction as the single primary endpoint; all cross-space/cross-load numbers are exploratory. Cost sensitivity to the illegal-trial penalty: reductions of 24.0% / 48.6% / 75.9% at $c_{\text{inv}} = 1/3/10$.

Two honest notes. (i) In the simulator’s small parameter subspace, arms M and A are *equivalent by construction*: the projected constraint is the same single bound, and **the effective constraint in this subspace is a single load-derived lower bound** (`mtib` ≥ 1024 , the trace’s practical prefill threshold)—RQ5 therefore quantifies the value of *having* that bound, while the value of *automatic* extraction is established by RQ1–RQ4, not by RQ5. (ii) The constraint’s *discrete* space reduction is only 12.3% (1.97M $\rightarrow$ 1.73M grid points), yet it removes about two-thirds of log-uniformly drawn samples before the optimizer adapts (31.5% charged-invalid): the value of a constraint depends on the sampler’s distribution, so both figures are reported. Figures 5–7 show cumulative cost, best-so-far goodput, and per-seed cost distributions.

Data-driven injection in the loop (six dimensions). We validate the generic sampler of §III-F end-to-end: the constraint graph JSON (716 nodes plus two auxiliary rules—one execution-discovered load bound and one simulator-derived KV-capacity rule, which is a performance/capacity rule rather than a legality constraint) is read directly and drives sampling in a six-dimensional space. Across 3 seeds $\times$ 30 trials = **90 real simulations, constraint violations and run failures are both 0/90, and cumulative cost equals the theoretical**

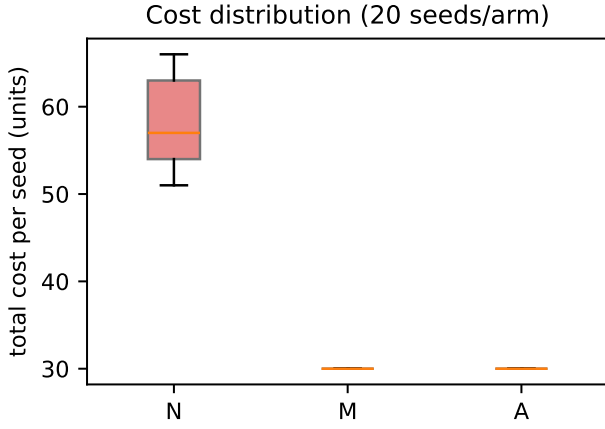


Fig. 7. Per-seed cost distributions (\$5).

TABLE VI

DIMENSION SCALING (QPS = 12, SEEDS 1–10); THE 6-D SPACE ADDS AN EXPLICIT NUM_BLOCKS DIMENSION.

Space	N cost	M/A cost	N→M/A reduction	N invalid
2-D	61.20 ± 2.90	30.00 ± 0.00	50.9% [49.3, 52.1]	35%
5-D (S5)	58.80 ± 5.33	30.00 ± 0.00	48.6% [45.9, 51.3]	32%
6-D	53.40 ± 4.43	30.00 ± 0.00	43.5% [40.9, 46.2]	26%

minimum (30.0 units); best goodput matches the hard-coded arm. This closes the loop from “constraint graph JSON” to tuner behaviour with no hand-written bounds.

Dimension scaling and workload robustness. Two further studies quantify how the benefit depends on the space and the load (same cost model; seeds 1–10; the 5-D row reuses the S5 runs).

The constrained arms stay at the theoretical minimum (30.0 units) in every space, while the unconstrained arm’s cost *falls* as dimensions grow (61.2 → 58.8 → 53.4 units): with more knobs the optimizer partially routes around violations, so the *relative* benefit is largest in small spaces but remains >40% even at 6-D. In the 6-D space the N arm violates the KV-capacity rule in **41.3%** of trials (124/300) versus 0% for the constrained arms. Workload robustness (5-D, qps ∈ {8, 12, 16}) gives reductions of **48.3%**, **48.6%**, and **49.6%**—stable across load regimes (Figure 8).

SCOOT-style online learning vs. automatic extraction. To isolate the value of *automatic* extraction from *online* learning, the **S** arm emulates SCOOT’s mechanism: only the manual rule $mtib \geq mns$ is enforced, and the lower bound is raised online whenever a trial fails (monotone boundary learning). Over 20 seeds, S costs **38.55 ± 2.80 units** (invalid rate 9.5%)—better than N (−33.5%) but **21.8% worse than the source-mined arms** (paired CI [19.3, 24.2]): online learning recovers most of the constraint surface, but it pays for the rediscovery in invalid trials, whereas extraction from source eliminates them from trial 1.

Real-engine A≠M micro-study (llama.cpp). RQ5’s main arms share the simulator’s constraint vocabulary; to test the distinction on a real engine against *folklore* rather than projected rules, we tuned llama.cpp v0.5.0 (Qwen2.5-

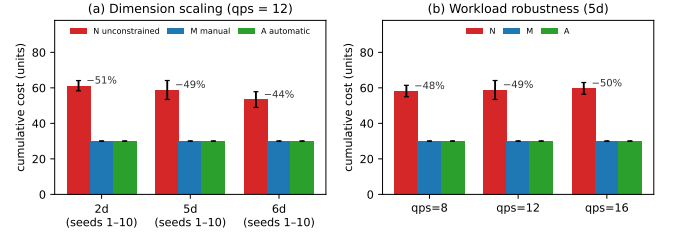


Fig. 8. Dimension scaling and workload robustness.

1.5B-Instruct, four-core CPU) over six dimensions—threads, n_batch , n_ubatch , flash attention, and K/V cache types (216 combinations; objective pp256). Four arms run 5 seeds × 20 trials each, interleaved trial-by-trial with a fixed calibration configuration to cancel the machine’s thermal drift: **N** no constraints; **M** the folk rule $n_ubatch \leq n_batch$ (refuted by our execution tests, §IV-C); **A** the source-mined, execution-confirmed constraint (quantized V-cache $\Rightarrow$ flash_attn; $n_batch \geq 1$ is non-binding in this space); and **S** a SCOOT-style arm that starts from M’s folk rule and *learns from failures*—after the first violating trial it forces attention on whenever V-cache quantization is sampled. **A incurs 0 of 100 invalid trials**; **S pays exactly one discovery failure per seed (5% total) before its learned rule takes over**; **M and N pay 12 and 17**—all 34 failures being precisely $ctv=q8_0$ with $fa=off$, the combination A forbids. **M and S never sample the legal $ubatch > batch$ region (0/88 and 0/95 vs 11/100 for A and 14/83 for N)**, which the engine accepts. Cumulative cost (success 1, failure 4 units): **A 20.0 ± 0.0 < S 23.0 ± 0.0 < M 27.2 ± 4.0 < N 30.2 ± 1.6**; paired gaps to A: −13.0% (S, **95% CI [3.0, 3.0] units**), −26.5% (M, [2.2, 12.2]), −33.8% (N, [8.2, 12.2]). The folk rule is simultaneously *incomplete* (it misses the constraint that fails) and *over-restrictive* (it forbids legal configurations); online learning recovers after one paid failure per study—the price of rediscovering what the source already states—whereas only source mining eliminates invalid trials from trial 1. **Final quality.** Re-running each arm’s best configuration with three repeats (rotated within seeds) shows no separation in attainable throughput—per-arm medians 48.5–63.3 t/s with fully overlapping ranges—on this machine the environment, not the arm, dominates absolute numbers; constraints change the *cost* of the search, not the peak it reaches.

Workload transfer. Repeating the full protocol on two workloads with far longer prefills—summarization (arxiv, prefill p50 ≈ 2.7K) and code (splitwise_code, p90 ≈ 5.2K)—gives N→M/A reductions of **46.2%** [43.6, 48.7] and **48.3%** [46.9, 49.8], with the constrained arms again at the theoretical minimum and the online-learning penalty stable at 23.1% and 20.3%. Notably, the load constraint’s empirical threshold is **trace-independent**: on all three traces, failures occur at $mtib \leq 1008$ and the smallest successful value is ≥ 1025 —a correctly mined constraint transfers across workloads without re-calibration.

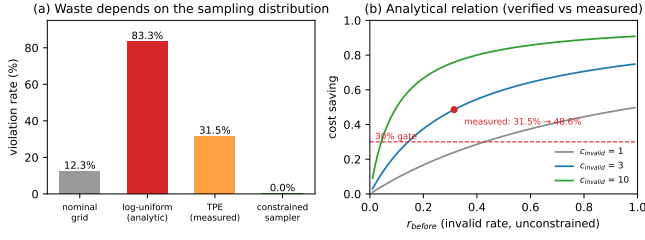


Fig. 9. Why the violation rate depends on the sampling distribution (left) and the analytical saving relation verified against the measured value (right).

TABLE VII
COST ACCOUNTING.

Item	Value
GPU hours	0 (vLLM validation is CPU; tuning is simulation; anchor is CPU llama.cpp)
Wall clock	extraction $\approx$ hours (agent time); falsification $\approx$ minutes; tuning $\approx$ 238 min (S5) + $\approx$ 5 h (dimension/workload) + 25 min (sampler) + $\approx$ 1.5 h (request-length workloads)
Cash	¥0
Simulation scale	8,000+ simulated trials (301 Optuna studies across arms/spaces/loads; real-hardware studies typically afford tens)

G. Cost accounting

V. RELATED WORK

Inference engine tuning. SCOOT [3] is the closest system: Bayesian optimization with hand-written known constraints and online-learned hidden constraints. We emulate its mechanism explicitly: its known-constraint component corresponds to our M arm and its online learning to our S arm. Across 20 seeds, the SCOOT-style arm costs 38.55 ± 2.80 units (9.5% invalid) versus 30.00 ± 0.00 for source-mined constraints—a **21.8% [19.3, 24.2] gap** representing the price of rediscovering, through invalid trials, constraints already stated in the source; and because two of SCOOT’s three published rules have since decayed (§II-D), its hand-written set does not cover the constraints that matter for this workload. We do not run SCOOT’s full implementation (its BO and crash-handling engineering are outside our emulation), so the comparison is mechanism-level. Our work supplies what SCOOT assumes: constraint *provenance* (source), *verification* (execution), and *maintenance* (re-extraction).

Configuration analysis and validation. Nadi et al. [25] pioneered mining configuration constraints from source—C code, via build-time errors and heuristics, for variability models; we revisit the problem with LLM agents and execution falsification for LLM engines. Dynamic invariant detection [26] infers likely invariants from executions rather than from source. cDep [18] statically extracts five types of configuration dependencies from Java/Scala bytecode and explicitly calls for NLP-based discovery of dependencies *between multiple parameters*—a call our LLM-based extraction

answers—but its output is dependency facts for developers, not executable pruning rules with execution-based verification. Configuration diagnosis [27] and white-box performance analysis [28] analyze deployed systems rather than mining constraints from source. PerfSense [16] and its journal successor CongSense [17] use LLM agents over call graphs and documentation to *classify* performance-sensitive parameters. Ciri [29] uses LLMs to *validate given configurations* (mis-configuration detection); it consumes configurations, whereas we mine constraints from source and inject them into a tuner. Configuration errors are a long-studied systems problem—surveys and diagnosis [30], [31], production configuration management [32], specification synthesis [33], and configuration testing [34]—but these lines validate, manage, or test *given* configurations rather than mining cross-parameter constraints for tuning. A parallel line infers constraints from logs (ConfInLog [13], ICPC’21) and applies configuration-aware static taint analysis for diagnosability (ConfTainter [14], ASE’23; ConfLogger, ICSE’26); these target misconfiguration diagnosis, not tuner-facing constraint sets. Finally, Pimp My LLM [15] treats LLM inference hyperparameters as a configurable system and manages constraints through a hand-built feature-based variability model—the closest neighbor on the inference-configuration side; unlike our pipeline, its constraints are neither mined from source nor verified by execution.

Knowledge-enhanced tuning. GPTuner [35] reads DBMS *manuals* to extract knob dependencies and value ranges for Bayesian optimization; LLMConf [36] uses a knowledge base to select parameters and their value ranges; LaTune [37] adapts tuning to edge devices via knowledge transfer; InferLog [38] tunes inference-engine configurations for log-parsing workloads. These systems rely on documentation or curated knowledge bases; we mine from *source* with verbatim evidence and verify by execution.

LLM agents for science and code. LLM agents are increasingly applied across software engineering [39]. POPPER [40] validates free-form hypotheses via sequential falsification with e-value error control; our falsification targets deterministic code constraints instead of statistical hypotheses.

Simulation. Vidur [19] predicts inference latency with $<9\%$ error and ships Vidur-Search, an enumerative configuration search whose space is human-defined; we use Vidur as an oracle and treat its hard-coded space as a contrast case.

Classical configuration tuning in databases, Spark, and compilers (e.g., database knob tuning [41], [42], [43], Rover [44], BaCO [45], HEBO [46]) shares the Bayesian machinery but not the source-mined constraint problem.

VI. LIMITATIONS AND THREATS TO VALIDITY

- 1) **Simulation fidelity.** RQ5 uses a validated simulator (Vidur, $<9\%$ error [19]) under vLLM-V0 semantics; V1-specific constraints do not participate. Relative comparisons are the paper’s currency; a GPU-based replication is future work.
- 2) **Offline-constructible subset.** 127 of the 282 core-tier constraints are runtime/request-time checks not exercised by construction-path falsification; they are marked,

not claimed. Cross-engine real execution partially compensates (llama.cpp).

- 3) **Anchor scale.** The llama.cpp evidence spans a 0.5B anchor grid and a 1.5B four-arm micro-study, both on a four-core CPU; absolute numbers do not represent GPU serving.
- 4) **Simulator artifact.** `block_size` is fixed to 16 because the bundled profiling data only supports 16; this is a simulator limitation, not a real-engine constraint.
- 5) **LLM dependence.** Extraction quality depends on the agent model and prompts—the general risks of LLM-based methods [47]; we mitigate with mechanical evidence checks, independent audits, an evidence-or-drop policy, and cross-model replication (97.2%).
- 6) **Version coverage.** Execution re-checks cover three versions (v0.5.5, v0.10.0, v0.30.0); systematic execution across all releases remains future work.
- 7) **Evaluator independence.** Audits and gold-standard labels were produced by pipeline roles and independent AI labeling passes rather than by external human experts; across five passes (60+60+30+150 AI sites, plus a 150-site human pass) they agree at $\kappa = 0.46$ –0.79 (human vs evaluator 0.53; human vs third pass 0.68), 23 of 24 disagreements in the first comparison run in the direction of the evaluator being more inclusive, and the stricter labels raise ± 1 -line recall to 92.3% on the batch-2 subset. One independent human labeler has now completed the 150-site kit pass; the full 300-site gold-standard frame and the 130 constraint audits remain AI-labeled, so a human sign-off for the complete release stays on the artifact checklist.
- 8) **Syntax coverage bias.** The headline gold-standard frame is the pipeline’s own candidate funnel (`assert/if-raise` sites mentioning an inventory parameter); other enforcement mechanisms (`argparse` checks, unguarded raises, `type/enum` annotations, warning-and-fallback branches, and the C++ engines’ internals) remain outside both numerator and denominator, so recall is conditional on the scanner’s syntactic coverage. The independent non-scanner frame in §IV-B bounds this dependence from a broader scope (68.0% at ± 1 line, vs. 87.2% on the candidate-pool frame; single adjudicator) and confirms the scope statement: most `assert/raise` sites in the tree are kernel-, tensor-, or checkpoint-level checks that the release does not claim to cover; the C++/non-Python surfaces remain unmeasured.
- 9) **Mechanism-level SCOOT comparison.** We emulate SCOOT’s known-constraint and online-learning components rather than running its full implementation; the 21.8% gap isolates the value of source mining, not the engineering quality of SCOOT. A head-to-head on GPUs is future work.

VII. CONCLUSION

Hand-written constraint knowledge decays: of the three rules a production tuning system ships, two no longer hold

in current releases, and none of six tested folk rules survives contact with a second engine. Yet the constraints themselves are abundant in the source. We showed how to mine them with evidence, falsify them by execution, and inject them into tuning—and we quantified the result: **716 evidence-backed constraints for vLLM and 420 for SGLang, with 113 confirmed against a real engine without any GPU**, three engines covered—and, across **8,000+ simulations**, a **43.5–50.9% cut in tuning cost** (headline 48.6%) with invalid sampling driven to zero. The artifacts are reproducible on a four-core laptop at zero cash cost.

VIII. ARTIFACT AVAILABILITY

The replication package (scripts, raw records, constraint graphs, and figures) is organized as follows and released with the paper:

- **Constraint graph (primary):** 282 constraints with evidence and validation status; frozen snapshot released.
- **Extended graph (full coverage):** **716** = 282 core + 434 from the full-coverage pass.
- **Second engine (SGLang):** **420** constraints (v0.5.20) with extraction artifacts.
- **Gold-standard recall:** the 300-site sample, per-site labels, and the combined results.
- **Longitudinal study:** six-version churn table and the rule-level re-verification records.
- **Verified set:** 113 confirmed (111 primary + 2 cross-model) plus cross-engine results (llama.cpp: 2 confirmed / 6 refuted).
- **Falsification:** 155 per-case records with captured engine error text.
- **Tuning studies:** S5 (three arms $\times$ 20 seeds), dimension scaling, arrival-rate and request-length workloads, sampler-in-loop, and the SCOOT-style arm — all Optuna studies, analysis CSVs, and reports.
- **Formalization and generic sampler:** constraint algebra, propositions, analytic relation, and the graph-driven sampler implementation.
- **Anchor:** llama.cpp constraint tests and 5-repetition performance benchmarks.
- **Scripts:** end-to-end extraction, scanning, merging, verification, tuning, analysis, and figure generation.

REFERENCES

- [1] W. Kwon, Z. Li, S. Zhuang, *et al.*, “Efficient memory management for large language model serving with PagedAttention,” in *Proc. SOSP*, 2023.
- [2] L. Zheng, L. Yin, Z. Xie, *et al.*, “SGLang: Efficient execution of structured language model programs,” in *Proc. NeurIPS*, 2024 (arXiv:2312.07104).
- [3] K. Cheng, Z. Wang, W. Hu, T. Yang, J. Li, and S. Zhang, “SCOOT: SLO-oriented performance tuning for LLM inference engines,” in *Proc. WWW*, 2025 (arXiv:2408.04323).
- [4] M. Liu, S. Zhong, W. Bi, *et al.*, “A first look at bugs in LLM inference engines,” *ACM Trans. Software Engineering and Methodology*, 2026 (arXiv:2506.09713).
- [5] llama.cpp. [Online]. Available: <https://github.com/ggml-org/llama.cpp>
- [6] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, “Orca: A distributed serving system for transformer-based generative models,” in *Proc. OSDI*, 2022.

- [7] A. Agrawal, N. Kedia, A. Panwar, J. Mohan, N. Kwatra, B. S. Gulavani, and R. Ramjee, "Taming throughput-latency tradeoff in LLM inference with Sarathi-Serve," in *Proc. OSDI*, 2024.
- [8] Y. Zhong, S. Liu, J. Chen, J. Hu, Y. Zhu, X. Liu, X. Jin, and H. Zhang, "DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving," in *Proc. OSDI*, 2024.
- [9] P. Moritz, R. Nishihara, S. Wang, *et al.*, "Ray: A distributed framework for emerging AI applications," in *Proc. OSDI*, 2018.
- [10] X. Miao, G. Oliaro, Z. Zhang, X. Cheng, H. Jin, T. Chen, and Z. Jia, "Towards efficient generative large language model serving: A survey from algorithms to systems," arXiv:2312.15234, 2023.
- [11] J. Snoek, H. Larochelle, and R. P. Adams, "Practical Bayesian optimization of machine learning algorithms," in *Proc. NeurIPS*, 2012.
- [12] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo, and D. Dennison, "Hidden technical debt in machine learning systems," in *Proc. NeurIPS*, 2015.
- [13] S.-L. Zhou, X. Liu, S. Li, Z.-Y. Jia, Y. Zhang, T. Wang, W. Li, and X. Liao, "ConflnLog: Leveraging software logs to infer configuration constraints," in *Proc. ICPC*, 2021, pp. 94–105.
- [14] T. Wang, H. He, X. Liu, S. Li, Z. Jia, Y. Jiang, Q. Liao, and W. Li, "Conftainter: Static taint analysis for configuration options," in *Proc. ASE*, 2023, pp. 1640–1651.
- [15] N. Zine, C. Quinton, and R. Rouvoy, "Pimp My LLM: Leveraging variability modeling to tune inference hyperparameters," arXiv:2602.17697, 2026.
- [16] Z. Wang, D. J. Kim, and T.-H. Chen, "Identifying performance-sensitive configurations in software systems through code analysis with LLM agents," arXiv:2406.12806, 2024.
- [17] Z. Wang, D. J. Kim, and T.-H. Chen, "Identifying performance-sensitive configurations in software systems with LLM-based agents (CongSense)," *Empirical Software Engineering*, vol. 31, art. 156, 2026.
- [18] Q. Chen, T. Wang, O. Legunsen, S. Li, and T. Xu, "Understanding and discovering software configuration dependencies in cloud and datacenter systems," in *Proc. ESEC/FSE*, 2020.
- [19] A. Agrawal, N. Kedia, J. Mohan, A. Panwar, N. Kwatra, B. S. Gulavani, R. Ramjee, and A. Tumanov, "Vidur: A large-scale simulation framework for LLM inference," in *Proc. MLSys*, 2024.
- [20] K. Claessen and J. Hughes, "QuickCheck: A lightweight tool for random testing of Haskell programs," in *Proc. ICFP*, 2000.
- [21] J. Bergstra, R. Bardenet, Y. Bengio, and B. Kégl, "Algorithms for hyperparameter optimization," in *Proc. NeurIPS*, 2011.
- [22] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," in *Proc. KDD*, 2019.
- [23] Y. Leviathan, M. Kalman, and Y. Matias, "Fast inference from transformers via speculative decoding," in *Proc. ICML*, 2023.
- [24] P. Patel, E. Choukse, C. Zhang, A. Shah, Íñigo Goiri, S. Maleki, and R. Bianchini, "Splitwise: Efficient generative LLM inference using phase splitting," in *Proc. ISCA*, 2024.
- [25] S. Nadi, T. Berger, C. Kästner, and K. Czarnecki, "Mining configuration constraints: Static analyses and empirical results," in *Proc. ICSE*, 2014.
- [26] M. D. Ernst, J. H. Perkins, P. J. Guo, S. McCamant, C. Pacheco, M. S. Tschantz, and C. Xiao, "The Daikon system for dynamic detection of likely invariants," *Science of Computer Programming*, vol. 69, no. 1–3, pp. 35–45, 2007.
- [27] Z. Chen, P. Chen, P. Wang, G. Yu, Z. He, and G. Mai, "DiagConfig: Configuration diagnosis of performance violations in configurable software systems," in *Proc. ESEC/FSE*, 2023.
- [28] M. Velez, P. Jamshidi, F. Sattler, N. Siegmund, S. Apel, and C. Kästner, "ConfigCrusher: Towards white-box performance analysis for configurable systems," *Automated Software Engineering*, vol. 27, pp. 265–300, 2020.
- [29] X. Lian, Y. Chen, R. Cheng, J. Huang, P. Thakkar, M. Zhang, T. Xu, and D. Marinov, "Large language models as configuration validators," in *Proc. ICSE*, 2025.
- [30] T. Xu and Y. Zhou, "Systems approaches to tackling configuration errors: A survey," *ACM Computing Surveys*, vol. 47, no. 4, art. 70, 2015.
- [31] Z. Yin, X. Ma, J. Zheng, Y. Zhou, L. N. Bairavasundaram, and S. Pasupathy, "Do not blame users for misconfigurations," in *Proc. SOSF*, 2013.
- [32] C. Tang, T. Kooburat, P. Venkatachalam, A. Chander, Z. Wen, A. Narayanan, P. Dowell, and R. Karl, "Holistic configuration management at Facebook," in *Proc. SOSF*, 2015.
- [33] M. Santolucito, E. Zhai, R. Dhodapkar, A. Shim, and R. Piskac, "Synthesizing configuration file specifications with association rule learning," *Proc. ACM Program. Lang. (OOPSLA)*, vol. 1, 2017.
- [34] S. Wang, X. Lian, Q. Li, D. Marinov, and T. Xu, "CTest4J: A practical configuration testing framework for Java," in *Proc. FSE (Demo)*, 2024.
- [35] J. Lao, Y. Wang, Y. Li, *et al.*, "GPTuner: A manual-reading database tuning system via GPT-guided Bayesian optimization," *PVLDB*, vol. 17, no. 8, 2024.
- [36] J. He, P. Chen, Y. Wang, H. Huang, C. Zhang, H. Huang, and D. Chen, "LLMConf: Knowledge-enhanced configuration optimization for large language model inference," in *Proc. IWQoS*, 2025.
- [37] S. Zhong, M. Liu, H. Shen, C. Pan, and Y. Ma, "LaTune: Lightweight and adaptive configuration tuning for LLM inference on edge devices," in *Proc. WWW*, 2026.
- [38] Y. Wang, P. Chen, H. Huang, Z. He, G. Tan, C. Zhang, J. He, and Z. Zheng, "InferLog: Accelerating LLM inference for online log parsing via ICL-oriented prefix caching," in *Proc. ICSE*, 2026.
- [39] X. Hou, Y. Zhao, Y. Liu, *et al.*, "Large language models for software engineering: A systematic literature review," *ACM Trans. Software Engineering and Methodology*, vol. 33, no. 8, 2024.
- [40] K. Huang, Y. Jin, R. Li, M. Y. Li, E. Candès, and J. Leskovec, "Automated hypothesis validation with agentic sequential falsifications," in *Proc. ICML*, 2025.
- [41] S. Duan, V. Thummala, and S. Babu, "Tuning database configuration parameters with iTuned," *PVLDB*, vol. 2, no. 1, pp. 1246–1257, 2009.
- [42] D. Van Aken, A. Pavlo, G. J. Gordon, and B. Zhang, "Automatic database management system tuning through large-scale machine learning," in *Proc. SIGMOD*, 2017.
- [43] K. Kanellis, C. Ding, B. Kroth, A. Müller, C. Curino, and S. Venkataraman, "LlamaTune: Sample-efficient DBMS configuration tuning," *PVLDB*, vol. 15, no. 11, pp. 2953–2965, 2022.
- [44] Y. Shen, X. Ren, Y. Lu, *et al.*, "Rover: An online Spark SQL tuning service via generalized transfer learning," in *Proc. KDD*, 2023.
- [45] E. O. Hellsten, A. Souza, J. Lenfers, *et al.*, "BaCO: A fast and portable Bayesian compiler optimization framework," in *Proc. ASPLOS*, 2023.
- [46] A. I. Cowen-Rivers, W. Lyu, R. Tutunov, *et al.*, "HEBO: Pushing the limits of sample-efficient hyperparameter optimisation," *JAIR*, vol. 74, pp. 1269–1349, 2022.
- [47] J. Sallou, T. Durieux, and A. Panichella, "Breaking the silence: The threats of using LLMs in software engineering," in *Proc. ICSE-NIER*, 2024.